Connecting VS Code to Your Lab VM

Remote-SSH · Supplement to Lab 0 · Revision 3

Tennessee Tech University · Department of Computer Science · Fall 2026

In Short

VS Code does not share a screen with your virtual machine. It opens an SSH connection, installs a small server program inside the VM, and runs it there. Your editor stays on your laptop; the files, the terminal, and the compiler all live in the guest.

Almost every failure you will hit is one of three things: the VM is not running an SSH server, your host cannot open a TCP connection to it, or the VM is too small to run the server process. The error messages do not distinguish between these, which is why this document works through them in order.

What You Should Be Able to Do

By the end of this supplement you should be able to reach a shell inside your VM from your own laptop, edit files in that VM from VS Code, open a file in the editor directly from the integrated terminal, and produce the diagnostic output needed to get help when a connection fails.

This guide covers **Windows and macOS hosts**, the two platforms that require a virtual machine for this course.

If your laptop already runs Linux

You do not need a VM and you do not need most of this guide. Install the course toolchain natively (`build-essential`, `python3`, `libssl-dev`, `tcpdump`/`tshark`), set up the same `~/csc4200-course` and `~/csc4200-work` folders Lab 0 describes, open the workspace in VS Code the ordinary way, and run the Lab 0 verification script directly. Section 10.4 and Section 11.6 still apply to you; the rest does not.

Contents

1 What you are actually building 4

2 Which path are you on? 4

3 Host prerequisites **5**
3.1 Windows . . . 5
3.2 macOS . . . 5

4 Install the Remote-SSH extension **6**

5 VirtualBox networking **6**
5.1 Option A — NAT with a port-forwarding rule (recommended) . . . 6
5.2 Option B — Host-only adapter . . . 7

6 UTM networking (alternate path) **7**
6.1 Option C — Shared Network (recommended for UTM) . . . 7
6.2 Option D — Emulated VLAN with port forwarding . . . 7

7 Guest (VM) configuration **8**
7.1 Install and start the SSH server . . . 8
7.2 Open the guest firewall . . . 8
7.3 Confirm the VM has room to work . . . 8
7.4 Note your VM username . . . 9

8 Test from a plain terminal first **9**

9 Set up key-based authentication **10**
9.1 Generate a key pair (on the host) . . . 10
9.2 Copy the public key into the VM . . . 10
9.3 Lock down the private key on the host . . . 10
9.4 Verify . . . 11

10 Configure and connect VS Code **11**
10.1 Write an SSH config entry . . . 11
10.2 Connect . . . 11
10.3 Install extensions on the remote side . . . 12
10.4 Sanity check . . . 12

11 Opening and editing files from the terminal **12**
11.1 The `code` command . . . 12
11.2 Everything is relative to the VM . . . 13
11.3 Going the other direction . . . 13
11.4 Files that need root . . . 13
11.5 When there is no `code` command . . . 14
11.6 Editing without VS Code . . . 14

12 Troubleshooting **14**
12.1 “Connection refused” . . . 14
12.2 “Connection timed out” or a long hang . . . 15
12.3 “Permission denied (publickey)” or repeating password prompts . . . 15
12.4 “REMOTE HOST IDENTIFICATION HAS CHANGED” . . . 15
12.5 VS Code connects, then drops, or retries forever . . . 15
12.6 “Server installation failed” or a stalled download . . . 16
12.7 “Server unsupported” or glibc errors . . . 16
12.8 Windows-only issues . . . 16
12.9 macOS-only issues . . . 16
12.10 It worked yesterday, not today . . . 16

13 Quick reference **17**

14 Getting help **17**

1 What you are actually building

VS Code does **not** share a screen with the VM. It opens an SSH connection, copies a small server program into the VM, and runs it there. Your editor window stays on your laptop; the files, the terminal, the compiler, and anything you execute all live inside the guest.

Three things must all be true, and a failure in any one of them produces an error message that does not tell you which:

1. The VM is running an SSH **server** (`sshd`).
2. Your host can open a TCP connection to that server. This is the part the hypervisor controls, and it is where most of you will get stuck.
3. The VM has enough RAM and disk to run the VS Code server process.



2 Which path are you on?

Your laptop	Hypervisor	Networking section
Windows 10/11 (Intel or AMD)	VirtualBox 7.2+	Section 5 — Option A or B
macOS on Intel	VirtualBox 7.2+	Section 5 — Option A or B
macOS on Apple Silicon (M1–M4)	VirtualBox 7.2+, Apple Silicon build	Section 5 — Option A or B
macOS on Apple Silicon, UTM fallback	UTM	Section 6 — Option C or D

Everything outside Sections 5 and 6 applies to all four rows. Only the networking step differs.

Currency Note: VirtualBox on Apple Silicon

VirtualBox runs natively on M-series Macs. Oracle added Apple Silicon host support in version 7.1 and now ships a dedicated Apple Silicon installer. Two consequences follow from the architecture, not from any immaturity in the software:

- Download the **Apple Silicon** package from the VirtualBox downloads page, not the Intel one. They carry the same version number and are separate files.
- An Apple Silicon host virtualizes **arm64** guests. Use the **arm64** Ubuntu 24.04 image. The `amd64` ISO is not a supported guest on this host.

Guest and host share the same instruction set here, so performance is good. This is native virtualization, not emulation.
Verified September 2026. Hypervisor platform support has roughly a two-to-three year accuracy half-life; re-check this note before each offering.

Windows: if the VM itself will not start

If VirtualBox reports `VERR_VMX_NO_VMX` or "VT-x is not available," Hyper-V or Memory Integrity is claiming the virtualization extensions before VirtualBox can. That is a Lab 0 setup issue, not an SSH issue. Resolve it first, then come back here.

Windows: if you chose the WSL2 path in Lab 0

Do not use Remote-SSH. Install the Microsoft **WSL** extension instead and connect with F1 → **WSL: Connect to WSL**. There is no VM, no port forwarding, and no SSH server involved, so Sections 5 to 9 do not apply to you. Pick up at Section 10.4.

3 Host prerequisites

3.1 Windows

Item	How to check	If missing
VS Code	**Help** → **About**	Install from `https://code.visualstudio.com`
OpenSSH client	`ssh -V` in **PowerShell**	**Settings** → **System** → **Optional features** → **Add an optional feature** → **OpenSSH Client**, then restart PowerShell
VirtualBox 7.2+	VirtualBox Manager → **Help** → **About**	Reinstall from `https://www.virtualbox.org`

Use **PowerShell**, not Command Prompt, for every host-side command in this guide. Open it with Win, type `powershell`, press Enter.

3.2 macOS

Item	How to check	If missing
VS Code	**VS Code** → **About Visual Studio Code**	Install from `https://code.visualstudio.com`
OpenSSH client	`ssh -V` in **Terminal**	Built in. If it fails, your `PATH` is broken
VirtualBox 7.2+	**VirtualBox** → **About VirtualBox**	Install from `https://www.virtualbox.org`, picking the package that matches your chip

Open **Terminal** with Cmd+Space, type `terminal`, press Enter.

macOS security prompt during VirtualBox install

macOS blocks VirtualBox's system extension on first install. Go to **System Settings** → **Privacy & Security**, scroll to the bottom, and click **Allow** next to the message about Oracle. You will need to reboot. If no **Allow** button appears, the extension was blocked silently: uninstall

VirtualBox, reinstall, and watch for the prompt.

4 Install the Remote-SSH extension

1. Open VS Code.
2. Open the Extensions panel: Ctrl+Shift+X on Windows, Cmd+Shift+X on macOS.
3. Search for **Remote - SSH**, publisher **Microsoft**. Install it.
4. A >< button appears in the bottom-left corner. That button is how you connect.

Do not substitute a third-party SSH extension. Only Microsoft's will install the VS Code server.

5 VirtualBox networking

Pick **one** option. Option A works identically on Windows and macOS and is what most of you should use.

5.1 Option A — NAT with a port-forwarding rule (recommended)

Your VM already uses NAT. NAT lets the VM reach the internet but gives the host no way to initiate a connection inward. A forwarding rule fixes that.

1. **Shut the VM down completely.** A real shutdown, not a saved state and not a pause.
2. VirtualBox Manager → select your VM → **Settings** → **Network**.
3. On the **Adapter 1** tab: **Enable Network Adapter** checked, **Attached to: NAT**.
4. Expand **Advanced** → click **Port Forwarding**.
5. Click **+** and fill the row exactly:

Name	Protocol	Host IP	Host Port	Guest IP	Guest Port
SSH	TCP	127.0.0.1	2222	*(leave blank)*	22

6. **OK** → **OK**. Start the VM.

Leave **Guest IP** blank; VirtualBox resolves it. Host port 2222 avoids the privileged-port restriction that would otherwise require administrator rights on both platforms. If something already occupies 2222, use 2223 and change it everywhere below.

Common Misconception: "My VM's address is the one `ip a` shows"

With Option A your VM's address *from the host* is `127.0.0.1` on port `2222`. It is **not** the `10.0.2.15` you see with `ip a` inside the VM. That address is NAT-internal and unreachable from the host by design.
This single misunderstanding causes more failed connections than everything else in this document combined.

5.2 Option B — Host-only adapter

Gives the VM a real address the host can reach without forwarding.

1. VirtualBox Manager → **Tools** → **Network** → **Host-only Networks**. If the list is empty, click **Create**. Note the range (usually `192.168.56.1/24`) and confirm DHCP is enabled.
2. Shut the VM down. **Settings** → **Network** → **Adapter 2**.
3. Check **Enable Network Adapter** → **Attached to: Host-only Adapter** → select the network from step 1.
4. **Leave Adapter 1 on NAT.** The VM still needs it for internet access; package installs and the VS Code server's first-run download both depend on it.
5. Start the VM and run `ip -4 addr show` inside it. Two addresses appear. The `192.168.56.x` one is what you use from the host, on port 22.

Do not use Bridged Adapter on campus

TTU's DHCP will often refuse the VM's second MAC address, and on the occasions it does work the address changes between sessions. Use Option A or Option B.

6 UTM networking (alternate path)

Skip this section if you are on VirtualBox.

6.1 Option C — Shared Network (recommended for UTM)

This is UTM's default and the simplest working configuration. No port forwarding is involved.

1. Shut the VM down. In UTM, select the VM → **Edit** (the settings slider icon) → **Network**.
2. Set **Network Mode: Shared Network**.
3. Save and start the VM.
4. In the VM's terminal, run `ip -4 addr show`. You will see an address in the `192.168.64.0/24` subnet.

Your VM's address from the host is that `192.168.64.x` value, on port 22. macOS can reach it directly.

The address is assigned by DHCP and can change after a reboot or a long shutdown. If a connection that worked yesterday times out today, re-check it. This is the most common UTM annoyance.

6.2 Option D — Emulated VLAN with port forwarding

Use this only if Option C is unavailable or blocked. It requires the **QEMU** backend, meaning **Use Apple Virtualization** must be **off** in the VM's **System** settings.

1. Shut the VM down. **Edit** → **Network** → **Network Mode: Emulated VLAN**.
2. Open the **Port Forward** tab → **New**:

Protocol	Guest Address	Guest Port	Host Address	Host Port
TCP	*(blank)*	22	127.0.0.1	2222

3. Save and start the VM.

Your VM's address from the host is `127.0.0.1` port `2222`, exactly as in Option A.

Currency Note: Guest image architecture under UTM

As with VirtualBox, use the **arm64** Ubuntu 24.04 image on Apple Silicon. UTM will *emulate* an amd64 ISO rather than refuse it, which is why a mismatched image shows up as a VM that boots but crawls.

7 Guest (VM) configuration

Run these **inside the VM**, in its own terminal window. Identical on every path above.

7.1 Install and start the SSH server

```
sudo apt update
sudo apt install -y openssh-server
sudo systemctl enable --now ssh
```

Checkpoint: is sshd listening?

```
sudo systemctl status ssh      # expect "active (running)"
ss -tlnp | grep :22            # expect 0.0.0.0:22 or *:22
```

If neither line looks like that, nothing later in this document will work.

7.2 Open the guest firewall

```
sudo ufw status
sudo ufw allow OpenSSH         # only if ufw reports "active"
```

7.3 Confirm the VM has room to work

```
free -h       # want ~1 GB available or more
df -h /       # want 5 GB free; the server unpacks into ~/.vscode-server
nproc         # 2 vCPU strongly recommended
```

If the VM was built with 1 GB of RAM, shut it down and raise it. In VirtualBox: **Settings** → **System** → **Motherboard** → **Base Memory** = 4096 MB, and **Processor** → 2 CPUs. In UTM: **Edit** → **System** → **Memory** = 4096 MB, **CPU Cores** = 2.

Practitioner Note: Rule this out early

A VM that boots perfectly at 1 GB will still fail to start the VS Code server. The symptom is a retry loop with no useful error, so it looks like a network problem and sends people down the wrong path for an hour.

7.4 Note your VM username

```
whoami
```

This is the VM account. Not your TTU username, not your laptop account. Use exactly this value everywhere below, where the examples say `yourvmuser`.

8 Test from a plain terminal first

Do this before opening VS Code

If SSH fails from a plain terminal it will fail in VS Code too, and the plain-SSH error message is far more legible. Every minute spent here saves several in Section 12.

Windows — PowerShell

```
# Option A or D (port forwarding)
ssh -p 2222 yourvmuser@127.0.0.1

# Option B (host-only)
ssh yourvmuser@192.168.56.101
```

macOS — Terminal

```
# Option A or D (port forwarding)
ssh -p 2222 yourvmuser@127.0.0.1

# Option B (host-only)
ssh yourvmuser@192.168.56.101

# Option C (UTM Shared Network)
ssh yourvmuser@192.168.64.5
```

Substitute the address you actually found. You should get a fingerprint prompt the first time — type `yes` — then a password prompt, then the VM's shell. Type `exit` to return.

If this fails, stop and work through Section 12 before going on.

9 Set up key-based authentication

Password authentication technically works, but VS Code opens several SSH channels and will prompt you over and over. Keys end that.

9.1 Generate a key pair (on the host)

Windows — PowerShell

```
ssh-keygen -t ed25519 -f "$env:USERPROFILE\.ssh\csc4200" -C "csc4200-vm"
```

macOS — Terminal

```
ssh-keygen -t ed25519 -f ~/.ssh/csc4200 -C "csc4200-vm"
```

Press Enter at the passphrase prompt to leave it empty, or set one — VS Code will ask for it once per session.

9.2 Copy the public key into the VM

Windows — PowerShell (one line; Windows has no ssh-copy-id)

```
Get-Content "$env:USERPROFILE\.ssh\csc4200.pub" | ssh -p 2222 yourvmuser@127.0.0.1
    ↪  "mkdir -p ~/.ssh && chmod 700 ~/.ssh && cat >> ~/.ssh/authorized_keys &&
    ↪ chmod 600 ~/.ssh/authorized_keys"
```

macOS — Terminal

```
ssh-copy-id -i ~/.ssh/csc4200.pub -p 2222 yourvmuser@127.0.0.1
```

For Options B and C, drop `-p 2222` and use the VM's address instead of `127.0.0.1`.

9.3 Lock down the private key on the host

Windows — OpenSSH refuses keys with loose ACLs

```
icacls "$env:USERPROFILE\.ssh\csc4200" /inheritance:r /grant:r "$($env:USERNAME):(
    ↪ R)"
```

macOS

```
chmod 600 ~/.ssh/csc4200
chmod 700 ~/.ssh
```

9.4 Verify

Checkpoint: passwordless login

```
ssh -i ~/.ssh/csc4200 -p 2222 yourvmuser@127.0.0.1
```

On Windows, use `-i "$env:USERPROFILE\.ssh\csc4200"`. You should land in the shell with no password prompt.

10 Configure and connect VS Code

10.1 Write an SSH config entry

This step makes everything afterward painless. In VS Code press F1 → **Remote-SSH: Open SSH Configuration File...** → choose the user-level file:

- **Windows:** `C:\Users\you\.ssh\config`
- **macOS:** `/Users/you/.ssh/config`

Add the block matching your option. Indentation matters — use spaces, not tabs.

Options A and D (port forwarding):

```
Host csc4200-vm
    HostName 127.0.0.1
    Port 2222
    User yourvmuser
    IdentityFile ~/.ssh/csc4200
```

Option B (VirtualBox host-only):

```
Host csc4200-vm
    HostName 192.168.56.101
    Port 22
    User yourvmuser
    IdentityFile ~/.ssh/csc4200
```

Option C (UTM shared network):

```
Host csc4200-vm
    HostName 192.168.64.5
    Port 22
    User yourvmuser
    IdentityFile ~/.ssh/csc4200
```

Windows users: if ~ gives trouble, write the full path instead.

```
    IdentityFile C:\Users\you\.ssh\csc4200
```

10.2 Connect

1. Make sure the VM is running.
2. F1 → **Remote-SSH: Connect to Host...** → **csc4200-vm**. Or click the >< button, bottom-left.

3. A new VS Code window opens. If it asks for the platform, choose **Linux**.
4. Watch the bottom-left indicator. It shows progress, then reads **SSH: csc4200-vm**. The first connection takes 30–90 seconds while the server downloads inside the VM.
5. **File** → **Open Folder** → `/home/yourvmuser/csc4200-work` → **Open**. That is your workspace, the folder Lab 0 set up for your own work. Do not open `csc4200-course` — those are course materials you only ever pull.
6. **Terminal** → **New Terminal**. This shell runs *in the VM*. Confirm with `hostname` and `uname -a`.

10.3 Install extensions on the remote side

Extensions are either host-side or remote-side, and all language tooling must be remote-side. With the remote window focused, open the Extensions panel. You will see a section headed **SSH: csc4200-vm — Installed**. For C/C++, Python, and anything else you need, click **Install in SSH: csc4200-vm**.

10.4 Sanity check

Checkpoint: the toolchain is where you think it is

In the remote terminal:

```
gcc --version
python3 --version
cd ~/csc4200-work && ls
```

You can also re-run the Lab 0 verification from here:

```
cd ~/csc4200-work
source venv/bin/activate
python3 ~/csc4200-course/lab0/verify_env.py
```

`lab0_report.txt` is written *inside the VM*, at `~/csc4200-work/lab0_report.txt` — that is the file to submit to Canvas.

11 Opening and editing files from the terminal

Once you are connected, the integrated terminal and the editor are two views of the same machine. The `code` command is what connects them.

11.1 The `code` command

The VS Code server installs a `code` shim on the VM's `PATH`. Running it from the **integrated terminal** opens the file in your editor — no file transfer, no second window, no copy on your laptop.

```
code notes.txt              # open a file in a new editor tab
code .                      # open the current directory as the workspace
code src/main.c include/    # open several files and folders at once
```

If the file does not exist, `code` creates an empty editor tab for it and writes it to the VM when you save.

Command	Effect
`code -r file`	Reuse the current window instead of opening a new one
`code -n file`	Force a new window
`code -g file:42`	Open at line 42
`code -g file:42:8`	Open at line 42, column 8
`code -d old.c new.c`	Open a side-by-side diff of two files
`code -w file`	Wait — the terminal blocks until you close the tab
`code -`	Read piped input into an untitled editor tab

Two of these are worth knowing well:

```
# Jump straight to a compiler error
gcc -Wall lab1.c -o lab1        # error reported at lab1.c:87
code -g lab1.c:87

# Capture a long build log in the editor instead of scrolling the terminal
make 2>&1 | code -
```

11.2 Everything is relative to the VM

Paths you give `code` are resolved on the **VM**, against the terminal's current directory — not against anything on your laptop. `pwd` tells you where you are. Tab completion works normally.

Because the file lives in the guest, saving with Ctrl+S or Cmd+S writes it to the VM's disk immediately. There is no sync step and nothing to copy back. Your laptop never holds a copy of the file.

11.3 Going the other direction

To get a terminal already sitting in the right directory: right-click a folder in the Explorer sidebar → **Open in Integrated Terminal**. New terminals otherwise open at the root of the workspace folder.

Ctrl+` (or Cmd+`) toggles the terminal panel. Ctrl+Shift+` opens an additional one, which is useful when you want a server running in one and a client in another for the socket assignments.

11.4 Files that need root

Never run sudo code

It will either refuse or run the server as root. The second outcome leaves root-owned files inside `~/.vscode-server`, which breaks your next connection with permission errors that are tedious to unpick.

For a configuration file such as `/etc/ssh/sshd_config`, use a terminal editor with `sudo`:

```
sudo nano /etc/ssh/sshd_config
sudo vim  /etc/ssh/sshd_config
```

For files in your own project directory you should never need `sudo` at all. If you do, something is owned by the wrong user. Fix the ownership rather than escalating:

```
sudo chown -R $USER:$USER ~/csc4200-work
```

If you have already run `sudo code` and connections started failing afterward, repair it from a plain SSH session:

```
sudo rm -rf ~/.vscode-server
```

Then reconnect; the server reinstalls cleanly.

11.5 When there is no `code` command

`code: command not found` means you are in a plain SSH session — a terminal you opened yourself with `ssh`, or a terminal window inside the VM's own console — rather than the VS Code integrated terminal. The shim only exists for sessions the server launched.

Open a terminal the right way with **Terminal** → **New Terminal** in the remote VS Code window, and confirm with:

```
which code                  # ~/.vscode-server/bin/<commit>/bin/remote-cli/code
echo $VSCODE_IPC_HOOK_CLI   # non-empty inside an integrated terminal
```

11.6 Editing without VS Code

You will still want these when you are debugging over plain SSH, or when the VS Code server will not start:

Command	Use
`nano file`	Simple editor. Ctrl+O writes, Ctrl+X exits
`vim file`	Modal editor. :wq writes and exits, :q! discards
`less file`	Page through a file without editing. q quits, / searches
`cat file`	Dump a short file to the terminal
`head -n 20 file`	First lines — useful on logs
`tail -n 20 file`	Last lines — useful on logs
`tail -f file`	Follow a file as it grows

`xdg-open file` hands a file to the desktop's default application, but only works on a desktop image with a graphical session attached. It does nothing useful over SSH.

12 Troubleshooting

Work top to bottom. Most failures are one of the first four.

12.1 "Connection refused"

Something reached the VM's port and actively said no.

- Is the VM genuinely running — booted to a login prompt, not paused or in a saved state?
- Is `sshd` up? In the VM: `sudo systemctl status ssh`. If not: `sudo systemctl enable --now ssh`.
- Did you add the port-forwarding rule **and then restart the VM**? Rules added while the VM runs do not always take effect until restart.
- Right port? Options A and D require `-p 2222`. Plain port 22 will be refused.

12.2 "Connection timed out" or a long hang

Nothing answered at all.

- **Wrong address.** With Option A or D the host address is `127.0.0.1`, not the `10.0.2.15` shown inside the VM.
- Option B or C: re-run `ip -4 addr show` in the VM. DHCP may have handed out a different address than last session.
- Guest firewall: `sudo ufw status`, then `sudo ufw allow OpenSSH` if active.
- **Windows:** third-party antivirus or endpoint software blocking loopback forwarding. Disable it temporarily to isolate.
- **macOS: System Settings** → **Network** → **Firewall**. If it is on, allow incoming connections for VirtualBox or UTM.

12.3 "Permission denied (publickey)" or repeating password prompts

- Permissions inside the VM must be exact. `sshd` silently ignores keys in a world-readable directory:

```
chmod 700 ~/.ssh
chmod 600 ~/.ssh/authorized_keys
chown -R $USER:$USER ~/.ssh
```

- Confirm the key arrived. In the VM, `cat` the file `~/.ssh/authorized_keys`; the last line should end with the comment `csc4200-vm`.
- Confirm `IdentityFile` points at the **private** key — no `.pub`.
- **Windows:** if you see "Permissions for ... are too open" or "bad permissions," run the `icacls` command from Section 9.3.
- A wrong username in the config is the most common cause of an endlessly repeating password prompt.

12.4 "REMOTE HOST IDENTIFICATION HAS CHANGED"

You reverted a snapshot or rebuilt the VM, so its host key is new. Clear the stale entry on the host:

```
ssh-keygen -R "[127.0.0.1]:2222"        # Options A, D
ssh-keygen -R 192.168.56.101            # Option B
ssh-keygen -R 192.168.64.5              # Option C
```

Reconnect and accept the new fingerprint.

12.5 VS Code connects, then drops, or retries forever

Nearly always a resource problem inside the VM.

- Check `free -h` and `df -h /`. The server wants roughly 500 MB of disk and 1 GB of available RAM. Raise the VM to 4096 MB if you have not (Section 7.3).
- Clear a half-installed server: in the VM, `rm -rf ~/.vscode-server`, then reconnect. Or from VS Code, F1 → **Remote-SSH: Kill VS Code Server on Host...**
- **Read the actual error:** F1 → **Remote-SSH: Show Log**, or the **Output** panel with **Remote - SSH** in the dropdown. The real cause is almost always in the last 20 lines.

12.6 "Server installation failed" or a stalled download

The VM needs working internet on first connect.

- In the VM: `ping -c 3 1.1.1.1` and `curl -I https://update.code.visualstudio.com`. If ping works but curl fails, DNS is misconfigured.
- Option B: confirm you kept **Adapter 1 on NAT**. A host-only adapter alone has no route out.
- On the campus network behind a proxy the download will be blocked. Use a phone hotspot for the first connection only.

12.7 "Server unsupported" or glibc errors

VS Code 1.86 and later require glibc 2.28 or newer. Ubuntu 18.04 and CentOS 7 images will fail. Rebuild from the **Ubuntu 24.04 LTS** image specified in Lab 0 — arm64 on Apple Silicon, amd64 everywhere else. Do not chase workarounds; the lab environment assumes 24.04.

12.8 Windows-only issues

- `ssh` not recognized → OpenSSH Client not installed (Section 3).
- VM will not start, `VERR_VMX_NO_VMX` → Hyper-V or Memory Integrity is holding the virtualization extensions. A Lab 0 issue; resolve it before returning here.
- Commands failing oddly → you are in Command Prompt, not PowerShell.

12.9 macOS-only issues

- VirtualBox installs but no VM will start → the system extension was blocked. **System Settings** → **Privacy & Security** → **Allow**, then reboot.
- Apple Silicon, the VM will not boot or the ISO is rejected → you downloaded the amd64 Ubuntu image. An Apple Silicon host runs arm64 guests; rebuild with the arm64 ISO.
- Apple Silicon, VirtualBox misbehaves after install → confirm you installed the **Apple Silicon** package rather than the Intel one. They are separate downloads with the same version number.
- UTM **Port Forward** tab missing → you are on the Apple Virtualization backend. Either switch to Shared Network (Option C) or turn off **Use Apple Virtualization** to expose Emulated VLAN.

12.10 It worked yesterday, not today

- The VM was restored from a saved state after a host reboot. Shut it down fully and cold-boot.
- Option B or C: the DHCP lease changed. Re-check the address.
- VirtualBox or UTM updated and reset the network configuration. Re-check Section 5 or Section 6.

13 Quick reference

Question	Answer or command
VM running?	VirtualBox or UTM shows **Running**
`sshd` running?	`sudo systemctl status ssh`
Listening?	`ss -tlnp | grep :22`
Address, Option A or D	`127.0.0.1` port `2222`
Address, Option B	`192.168.56.x` port `22`
Address, Option C	`192.168.64.x` port `22`
Reachable?	`ssh -p <port> user@<address>`
Key permissions, guest	`chmod 700 ~/.ssh ; chmod 600 ~/.ssh/authorized_keys`
Key permissions, Windows	`icacls ... /inheritance:r /grant:r`
Connect	F1 → **Remote-SSH: Connect to Host** → **csc4200-vm**
Real error message	F1 → **Remote-SSH: Show Log**
Reset the remote server	F1 → **Remote-SSH: Kill VS Code Server on Host**
Open a file from terminal	`code file` or `code -g file:42`
Never	`sudo code`

14 Getting help

Bring these five items to office hours, or post them with your question in Teams:

1. Your host OS and chip — Windows, Intel Mac, or Apple Silicon Mac.
2. Which option you configured — A, B, C, or D.
3. The full output of a verbose SSH attempt from your host, adjusting the address and port to match your option. The `-vvv` is what makes this useful:

```
ssh -vvv -p 2222 yourvmuser@127.0.0.1
```

4. Output of `sudo systemctl status ssh` and `ss -tlnp | grep :22` from inside the VM.
5. The last 20 lines of **Remote-SSH: Show Log**.

Practitioner Note: What makes a report diagnosable

A screenshot of the failure dialog alone is not diagnosable — every one of the failures in Section 12 produces the same dialog. The verbose SSH output almost always is, because it shows which of the three preconditions in Section 2 actually broke.